

Hướng dẫn học ROS2: Giai đoạn 1 (Core Concepts) & Giai đoạn 2 (TF2)

Giả định: bạn dùng ROS2 Humble trên Ubuntu 22.04, đã cài xong ROS2 và tạo workspace (`~/ros2_ws/src`).

GIAI ĐOẠN 1: CORE CONCEPTS

Chuẩn bị workspace

```
bash
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_python my_pubsub \
--dependencies rclpy std_msgs
```

Cấu trúc sinh ra:

```
my_pubsub/
├── my_pubsub/      # thư mục Python chứa code node
│   └── __init__.py
├── package.xml
├── setup.py
├── setup.cfg
└── resource/my_pubsub
```

Ghi nhớ vòng đời làm việc: viết code → sửa `setup.py` (entry_points) → `colcon build` ở root workspace → `source install/setup.bash` → `ros2 run`. Đây là chu trình bạn sẽ lặp lại hàng chục lần, hãy thuộc lòng nó chứ đừng chỉ đọc.

1. Node + Topic (Pub/Sub)

Khái niệm cốt lõi

Node: một tiến trình độc lập, làm một việc cụ thể (đọc cảm biến, điều khiển động cơ, xử lý ảnh...). Một robot thật có thể có hàng chục node chạy song song.

Topic: kênh giao tiếp bất đồng bộ, kiểu **publish/subscribe** — publisher không biết ai đang nghe, subscriber không biết ai đang gửi. Đây là mô hình **loose coupling** rất quan trọng của ROS.

Giao tiếp qua topic dùng **message type** cố định (ví dụ `std_msgs/msg/Int32`), không tự do định dạng.

Dưới nền là DDS (Data Distribution Service) lo việc discovery và truyền dữ liệu — bạn không cần biết sâu DDS ở giai đoạn này, chỉ cần biết ROS2 khác ROS1 ở chỗ không có "master" trung tâm.

Viết Publisher (`talker.py`)

Tạo file `my_pubsub/my_pubsub/talker.py`:

```
python
```

```

import rclpy
from rclpy.node import Node
from std_msgs.msg import Int32

class CounterPublisher(Node):
    def __init__(self):
        super().__init__('counter_publisher')
        self.publisher_ = self.create_publisher(Int32, 'counter', 10)
        self.timer = self.create_timer(1.0, self.timer_callback) # 1 giây/lần
        self.count = 0

    def timer_callback(self):
        msg = Int32()
        msg.data = self.count
        self.publisher_.publish(msg)
        self.get_logger().info(f'Publishing: {msg.data}')
        self.count += 1

def main(args=None):
    rclpy.init(args=args)
    node = CounterPublisher()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

Điểm cần hiểu, không chỉ copy:

`create_publisher(MsgType, topic_name, qos)` — số `10` là **queue size** (QoS đơn giản hoá), tạm hiểu là hàng đợi buffer.

`create_timer` tạo callback định kỳ, chạy trong **executor** khi bạn gọi `rclpy.spin()`.

`rclpy.spin(node)` là vòng lặp chặn (blocking), giữ node sống và xử lý callback.

Viết Subscriber (`listener.py`)

`my_pubsub/my_pubsub/listener.py`:

python

```

import rclpy
from rclpy.node import Node
from std_msgs.msg import Int32

class CounterSubscriber(Node):
    def __init__(self):
        super().__init__('counter_subscriber')
        self.subscription = self.create_subscription(
            Int32, 'counter', self.listener_callback, 10)

    def listener_callback(self, msg):
        self.get_logger().info(f'Received: {msg.data}')

def main(args=None):
    rclpy.init(args=args)
    node = CounterSubscriber()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

Đăng ký entry point

Sửa `setup.py`, thêm vào `entry_points`:

```

python
entry_points={
    'console_scripts': [
        'talker = my_pubsub.talker:main',
        'listener = my_pubsub.listener:main',
    ],
},

```

Build và chạy

```

bash
cd ~/ros2_ws
colcon build --packages-select my_pubsub
source install/setup.bash

```

Mở 2 terminal (mỗi terminal nhớ `source install/setup.bash`):

```
bash
# Terminal 1
ros2 run my_pubsub talker
```

```
# Terminal 2
ros2 run my_pubsub listener
```

Học các lệnh CLI (rất quan trọng, đừng bỏ qua)

```
bash
ros2 topic list      # liệt kê tất cả topic đang hoạt động
ros2 topic echo /counter # in dữ liệu topic ra terminal (không cần listener.py)
ros2 topic hz /counter  # đo tần số publish thực tế
ros2 topic info /counter # xem type, số publisher/subscriber
ros2 topic pub /counter std_msgs/msg/Int32 "{data: 5}" # publish thủ công từ CLI
```

Bài tập tự làm: đổi publisher gửi 2 giá trị (dùng message tự định nghĩa hoặc `geometry_msgs/msg/Point`), viết subscriber in ra dạng "x=.. y=..". Việc tự viết message custom (`.msg` file + `rosidl_generate_interfaces`) là bước nên thử ở cuối tuần 1 nếu còn thời gian.

2. Service (Request/Response)

Khi nào dùng Service thay vì Topic

Topic: dữ liệu liên tục, không cần phản hồi (cảm biến, trạng thái).

Service: **đồng bộ**, một yêu cầu — một phản hồi, xong thì kết thúc (tính toán, truy vấn, bật/tắt một chế độ).

Service **chặn** (blocking) cho tới khi có kết quả.

Tạo package mới cho service

```
bash
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_python my_service \
--dependencies rclpy example_interfaces
```

Ta dùng sẵn `example_interfaces/srv/AddTwoInts` thay vì tự định nghĩa `.srv` (để bớt phức tạp ở bước đầu).

Service Server

```
my_service/my_service/add_server.py:
```

```
python
```

```

import rclpy
from rclpy.node import Node
from example_interfaces.srv import AddTwoInts

class AddTwoIntsServer(Node):
    def __init__(self):
        super().__init__('add_two_ints_server')
        self.srv = self.create_service(
            AddTwoInts, 'add_two_ints', self.add_callback)

    def add_callback(self, request, response):
        response.sum = request.a + request.b
        self.get_logger().info(
            f'Request: {request.a} + {request.b} = {response.sum}')
        return response

def main(args=None):
    rclpy.init(args=args)
    node = AddTwoIntsServer()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

Service Client

`my_service/my_service/add_client.py`:

python

```

import sys
import rclpy
from rclpy.node import Node
from example_interfaces.srv import AddTwoInts

class AddTwoIntsClient(Node):
    def __init__(self):
        super().__init__('add_two_ints_client')
        self.client = self.create_client(AddTwoInts, 'add_two_ints')
        while not self.client.wait_for_service(timeout_sec=1.0):
            self.get_logger().info('Waiting for service...')

    def send_request(self, a, b):
        req = AddTwoInts.Request()
        req.a = a
        req.b = b
        future = self.client.call_async(req)
        rclpy.spin_until_future_complete(self, future)
        return future.result()

def main(args=None):
    rclpy.init(args=args)
    node = AddTwoIntsClient()
    a, b = int(sys.argv[1]), int(sys.argv[2])
    result = node.send_request(a, b)
    node.get_logger().info(f'Result: {result.sum}')
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

Điểm quan trọng: `call_async` + `spin_until_future_complete` là cách gọi service **không bị deadlock**. Nếu bạn gọi `.result()` trực tiếp mà không spin, node sẽ treo — đây là lỗi rất phổ biến với người mới.

Đăng ký entry points tương tự phần 1, rồi build:

```

bash
colcon build --packages-select my_service
source install/setup.bash
ros2 run my_service add_server    # terminal 1
ros2 run my_service add_client 3 5 # terminal 2

```

CLI cho Service

```
bash
ros2 service list
ros2 service type /add_two_ints
ros2 service call /add_two_ints example_interfaces/srv/AddTwoInts "{a: 2, b: 3}"
```

3. Action (Long-running task có feedback)

Khi nào dùng Action

Dùng Action khi task **kéo dài** (vài giây đến vài phút), cần **feedback theo tiến trình**, và có thể **huỷ giữa chừng**. Ví dụ thực tế: robot di chuyển đến một điểm, tay robot thực hiện một chuỗi thao tác, quá trình sạc pin.

So sánh nhanh:

	Topic	Service	Action
Kiểu	Liên tục	1 request - 1 response	Goal - Feedback... - Result
Chặn?	Không	Có (đồng bộ)	Không (bất đồng bộ), có preemption
Huỷ giữa chừng	Không áp dụng	Không	Có
Ví dụ	Dữ liệu lidar	Cộng 2 số	Di chuyển robot tới điểm X

Tạo custom action

Action cần **package interface riêng** (không thể để chung package Python thông thường vì cần build bằng CMake/rosidl). Tạo package interface:

```
bash
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_cmake my_action_interfaces
mkdir my_action_interfaces/action
```

Tạo file `my_action_interfaces/action/CountUntil.action`:

```
# Goal
int32 target_number
float32 period    # thời gian giữa mỗi bước đếm (giây)
---
# Result
int32 reached_number
---
# Feedback
int32 current_number
```

Sửa `CMakeLists.txt` của `my_action_interfaces`, thêm trước `ament_package()`:

cmake

```
find_package(rosidl_default_generators REQUIRED)
```

```
rosidl_generate_interfaces(${PROJECT_NAME}
```

```
"action/CountUntil.action"
```

```
)
```

Và trong `package.xml` thêm:

xml

```
<buildtool_depend>rosidl_default_generators</buildtool_depend>
```

```
<exec_depend>rosidl_default_runtime</exec_depend>
```

```
<member_of_group>rosidl_interface_packages</member_of_group>
```

Build package interface riêng trước:

bash

```
cd ~/ros2_ws
```

```
colcon build --packages-select my_action_interfaces
```

```
source install/setup.bash
```

Action Server

Package Python: `ros2 pkg create --build-type ament_python my_action --dependencies rclpy`

```
my_action_interfaces rclpy.action
```

```
my_action/my_action/count_server.py:
```

python


```
import time
import rclpy
from rclpy.action import ActionServer, CancelResponse, GoalResponse
from rclpy.node import Node
from my_action_interfaces.action import CountUntil
```

```
class CountUntilServer(Node):
```

```
    def __init__(self):
        super().__init__('count_until_server')
        self._action_server = ActionServer(
            self,
            CountUntil,
            'count_until',
            execute_callback=self.execute_callback,
            goal_callback=self.goal_callback,
            cancel_callback=self.cancel_callback)
```

```
    def goal_callback(self, goal_request):
        self.get_logger().info('Received goal request')
        return GoalResponse.ACCEPT
```

```
    def cancel_callback(self, goal_handle):
        self.get_logger().info('Received cancel request')
        return CancelResponse.ACCEPT
```

```
    def execute_callback(self, goal_handle):
        target = goal_handle.request.target_number
        period = goal_handle.request.period
        feedback_msg = CountUntil.Feedback()
        counter = 0
```

```
        for i in range(target):
            if goal_handle.is_cancel_requested:
                goal_handle.canceled()
                self.get_logger().info('Goal canceled')
                result = CountUntil.Result()
                result.reached_number = counter
                return result

            counter += 1
            feedback_msg.current_number = counter
            goal_handle.publish_feedback(feedback_msg)
            self.get_logger().info(f'Feedback: {counter}')
```

```
time.sleep(period)
```

```
goal_handle.succeed()
```

```
result = CountUntil.Result()
```

```
result.reached_number = counter
```

```
return result
```

```
def main(args=None):
```

```
    rclpy.init(args=args)
```

```
    node = CountUntilServer()
```

```
    rclpy.spin(node)
```

```
    node.destroy_node()
```

```
    rclpy.shutdown()
```

```
if __name__ == '__main__':
```

```
    main()
```

Action Client

```
my_action/my_action/count_client.py:
```

```
python
```

```

import rclpy
from rclpy.action import ActionClient
from rclpy.node import Node
from my_action_interfaces.action import CountUntil

class CountUntilClient(Node):
    def __init__(self):
        super().__init__('count_until_client')
        self._action_client = ActionClient(self, CountUntil, 'count_until')

    def send_goal(self, target, period):
        goal_msg = CountUntil.Goal()
        goal_msg.target_number = target
        goal_msg.period = period

        self._action_client.wait_for_server()
        self._send_goal_future = self._action_client.send_goal_async(
            goal_msg, feedback_callback=self.feedback_callback)
        self._send_goal_future.add_done_callback(self.goal_response_callback)

    def goal_response_callback(self, future):
        goal_handle = future.result()
        if not goal_handle.accepted:
            self.get_logger().info('Goal rejected')
            return
        self.get_logger().info('Goal accepted')
        self._get_result_future = goal_handle.get_result_async()
        self._get_result_future.add_done_callback(self.get_result_callback)

    def get_result_callback(self, future):
        result = future.result().result
        self.get_logger().info(f'Result: {result.reached_number}')
        rclpy.shutdown()

    def feedback_callback(self, feedback_msg):
        self.get_logger().info(f'Feedback: {feedback_msg.feedback.current_number}')

def main(args=None):
    rclpy.init(args=args)
    node = CountUntilClient()
    node.send_goal(5, 1.0)
    rclpy.spin(node)

```

```
if __name__ == '__main__':  
    main()
```

CLI cho Action

```
bash  
ros2 action list  
ros2 action info /count_until  
ros2 action send_goal /count_until my_action_interfaces/action/CountUntil "{target_number: 5, period: 1.0}"
```

Thêm `Ctrl+C` khi đang chạy `send_goal` để test hành vi cancel.

Bài tập: thử sửa action để robot giả lập "di chuyển đến vị trí X", feedback là khoảng cách còn lại, cancel giữa chừng và kiểm tra `reached_number` trả về đúng giá trị dừng.

4. Parameters + Launch files

Parameters

Parameter cho phép cấu hình node **lúc chạy** mà không sửa code — ví dụ tốc độ tối đa, tên topic, ngưỡng cảm biến.

```
python
```

```

import rclpy
from rclpy.node import Node

class ParamDemoNode(Node):
    def __init__(self):
        super().__init__('param_demo_node')
        self.declare_parameter('robot_name', 'default_robot')
        self.declare_parameter('max_speed', 1.0)

        name = self.get_parameter('robot_name').get_parameter_value().string_value
        speed = self.get_parameter('max_speed').get_parameter_value().double_value
        self.get_logger().info(f'{name} chạy với max_speed={speed}')

        # callback khi param bị đổi lúc runtime
        self.add_on_set_parameters_callback(self.param_callback)

    def param_callback(self, params):
        from rcl_interfaces.msg import SetParametersResult
        for param in params:
            self.get_logger().info(f'Param {param.name} đổi thành {param.value}')
        return SetParametersResult(successful=True)

def main(args=None):
    rclpy.init(args=args)
    node = ParamDemoNode()
    rclpy.spin(node)

if __name__ == '__main__':
    main()

```

CLI:

```

bash
ros2 param list
ros2 param get /param_demo_node max_speed
ros2 param set /param_demo_node max_speed 2.5

```

Chạy với param từ đầu:

```

bash
ros2 run my_pubsub param_demo --ros-args -p robot_name:=my_bot -p max_speed:=3.0

```

Hoặc file YAML (`config/params.yaml`):

```
yaml
```

```
param_demo_node:
  ros__parameters:
    robot_name: "my_bot"
    max_speed: 3.0
```

```
bash
```

```
ros2 run my_pubsub param_demo --ros-args --params-file config/params.yaml
```

Launch files

Launch file cho phép chạy **nhiều node cùng lúc**, kèm param, remap, namespace — thứ bạn sẽ dùng liên tục khi hệ thống lớn dần.

```
my_pubsub/launch/pubsub_launch.py:
```

```
python
```

```
from launch import LaunchDescription
from launch_ros.actions import Node
```

```
def generate_launch_description():
    return LaunchDescription([
        Node(
            package='my_pubsub',
            executable='talker',
            name='counter_publisher',
            output='screen',
        ),
        Node(
            package='my_pubsub',
            executable='listener',
            name='counter_subscriber',
            output='screen',
        ),
    ])
```

Thêm vào `setup.py` để launch file được cài đặt đúng chỗ (trong `data_files`):

```
python
```

```
import os
from glob import glob

data_files=[
    ...
    (os.path.join('share', package_name, 'launch'), glob('launch/*_launch.py')),
],
```

Build và chạy:

```
bash
colcon build --packages-select my_pubsub
source install/setup.bash
ros2 launch my_pubsub pubsub_launch.py
```

Bài tập: thêm param `count_interval` vào talker, truyền qua launch file bằng `parameters=[('count_interval': 0.5)]`, và thử remap topic name bằng `remappings=[('counter', 'my_counter')]`.

📌 Tài liệu chuẩn: theo sát [ROS2 Humble Tutorials](#), làm tay từng bài, đừng chỉ đọc.

GIAI ĐOẠN 2: TF2 (TRANSFORM)

TF2 là phần hay gây rối vì nó trừu tượng (toạ độ, phép quay) nhưng lại là **nền tảng bắt buộc** cho URDF, Nav2, MoveIt — mọi thứ liên quan không gian 3D trong ROS2 đều đi qua TF.

Khái niệm cốt lõi

Frame là gì

Một **frame** (khung toạ độ) là một hệ trục toạ độ gắn với một vật thể hoặc điểm tham chiếu: gắn trên thân robot, trên bánh xe, trên camera, hoặc là gốc toạ độ bản đồ. Mỗi frame có tên riêng, ví dụ `base_link`, `camera_link`, `map`.

Transform là gì

Một **transform** mô tả **vị trí + hướng (pose)** của một frame so với frame khác — gồm translation (x, y, z) và rotation (thường là quaternion trong ROS2, không phải Euler).

Transform Tree

TF lưu tất cả transform thành một **cây** (tree), không phải đồ thị tùy ý:

Mỗi frame chỉ có **đúng một cha** (parent), nhưng có thể có nhiều con.

Nếu 2 node cùng publish transform cho cùng 1 frame với 2 cha khác nhau → lỗi "multiple parents", TF sẽ báo lỗi hoặc dữ liệu sai.

Muốn biết pose của frame A so với frame B, TF sẽ tự tìm đường đi trong cây và nhân các transform lại (kể cả khi A và B không liên hệ trực tiếp).

Các frame chuẩn (theo REP 105)

```
map → odom → base_link → (base_footprint, sensor frames, camera_link, laser_frame...)
```

`map`: cố định tuyệt đối, gốc toạ độ toàn cục (dùng cho localization dài hạn, có thể nhảy bậc khi sửa lỗi định vị).

`odom`: cố định tương đối, liên tục mượt mà nhưng có thể trôi (drift) theo thời gian — lấy từ odometry (encoder bánh xe, IMU).

`base_link`: gắn liền vào thân robot — mọi sensor frame là con của nó.

Quan hệ `odom → base_link` thường do node điều khiển/odometry publish liên tục (dynamic), còn `map → odom` do hệ thống localization (AMCL, SLAM) publish.

Hiểu đúng 3 tầng này là **chìa khoá** để sau này không rối khi học Nav2.

Static Transform Broadcaster

Dùng khi transform **không đổi theo thời gian** — ví dụ camera gắn cố định trên khung robot.

Cách nhanh nhất: dùng node có sẵn (không cần code)

```
bash
ros2 run tf2_ros static_transform_publisher \
  --x 0.1 --y 0 --z 0.2 \
  --roll 0 --pitch 0 --yaw 0 \
  --frame-id base_link --child-frame-id camera_link
```

Nghĩa: `camera_link` cách `base_link` 0.1m theo x, 0.2m theo z, không xoay.

Cách viết bằng code (để hiểu cơ chế)

`my_tf/my_tf/static_broadcaster.py`:

```
python
```



```

import rclpy
from rclpy.node import Node
from tf2_ros import StaticTransformBroadcaster
from geometry_msgs.msg import TransformStamped

class StaticFramePublisher(Node):
    def __init__(self):
        super().__init__('static_broadcaster')
        self.broadcaster = StaticTransformBroadcaster(self)
        self.make_transform()

    def make_transform(self):
        t = TransformStamped()
        t.header.stamp = self.get_clock().now().to_msg()
        t.header.frame_id = 'base_link'
        t.child_frame_id = 'camera_link'
        t.transform.translation.x = 0.1
        t.transform.translation.y = 0.0
        t.transform.translation.z = 0.2
        t.transform.rotation.x = 0.0
        t.transform.rotation.y = 0.0
        t.transform.rotation.z = 0.0
        t.transform.rotation.w = 1.0 # quaternion "không xoay"
        self.broadcaster.sendTransform(t)

def main(args=None):
    rclpy.init(args=args)
    node = StaticFramePublisher()
    rclpy.spin(node)

if __name__ == '__main__':
    main()

```

Lưu ý: `StaticTransformBroadcaster` chỉ gửi **1 lần** nhưng publish trên topic `/tf_static` với QoS "latched" (transient local) nên node mới join sau vẫn nhận được — khác với `/tf` thông thường phải gửi liên tục.

Dynamic Transform Broadcaster

Dùng khi transform **thay đổi liên tục** — ví dụ robot di chuyển, cần cập nhật `odom → base_link` mỗi vòng lặp.

`my_tf/my_tf/dynamic_broadcaster.py` — giả lập robot đi vòng tròn:

python

```

import math
import rclpy
from rclpy.node import Node
from tf2_ros import TransformBroadcaster
from geometry_msgs.msg import TransformStamped

class DynamicFramePublisher(Node):
    def __init__(self):
        super().__init__('dynamic_broadcaster')
        self.broadcaster = TransformBroadcaster(self)
        self.timer = self.create_timer(0.1, self.broadcast_timer_callback)
        self.t = 0.0

    def broadcast_timer_callback(self):
        t = TransformStamped()
        t.header.stamp = self.get_clock().now().to_msg()
        t.header.frame_id = 'odom'
        t.child_frame_id = 'base_link'

        radius = 1.0
        t.transform.translation.x = radius * math.cos(self.t)
        t.transform.translation.y = radius * math.sin(self.t)
        t.transform.translation.z = 0.0

        # quaternion cho hướng robot (yaw = self.t)
        t.transform.rotation.z = math.sin(self.t / 2.0)
        t.transform.rotation.w = math.cos(self.t / 2.0)

        self.broadcaster.sendTransform(t)
        self.t += 0.05

def main(args=None):
    rclpy.init(args=args)
    node = DynamicFramePublisher()
    rclpy.spin(node)

if __name__ == '__main__':
    main()

```

Chạy song song cả static và dynamic broadcaster, rồi kiểm tra:



```
bash
```

```
ros2 run tf2_ros tf2_echo odom base_link # in transform theo thời gian thực
```

```
ros2 run tf2_ros tf2_echo base_link camera_link
```

Lắng nghe transform (TransformListener) — phần hay bị bỏ sót

Publish transform mới chỉ là một nửa. Muốn **dùng** transform (ví dụ chuyển tọa độ một điểm từ `camera_link` sang `base_link`), bạn cần `Buffer` + `TransformListener`:

```
python
```

```
import rclpy
```

```
from rclpy.node import Node
```

```
from tf2_ros import Buffer, TransformListener
```

```
from tf2_ros import LookupException, ConnectivityException, ExtrapolationException
```

```
class TfListenerNode(Node):
```

```
    def __init__(self):
```

```
        super().__init__('tf_listener_node')
```

```
        self.tf_buffer = Buffer()
```

```
        self.tf_listener = TransformListener(self.tf_buffer, self)
```

```
        self.timer = self.create_timer(1.0, self.on_timer)
```

```
    def on_timer(self):
```

```
        try:
```

```
            t = self.tf_buffer.lookup_transform(
```

```
                'base_link', 'camera_link', rclpy.time.Time())
```

```
            self.get_logger().info(
```

```
                f'x={t.transform.translation.x:.2f}, '
```

```
                f'y={t.transform.translation.y:.2f}')
```

```
        except (LookupException, ConnectivityException, ExtrapolationException) as e:
```

```
            self.get_logger().warn(f'Could not transform: {e}')
```

```
def main(args=None):
```

```
    rclpy.init(args=args)
```

```
    rclpy.spin(TfListenerNode())
```

```
if __name__ == '__main__':
```

```
    main()
```

Công cụ trực quan hoá

```
bash
```

`ros2 run tf2_tools view_frames`

→ Sinh ra file `frames.pdf` vẽ toàn bộ cây transform hiện có — cực hữu ích để debug khi hệ thống nhiều frame.

Trong RViz2:

```
bash
rviz2
```

Add → By display type → **TF** → sẽ thấy trục tọa độ (RGB = x/y/z) của từng frame và các đường nối thể hiện cây transform.

Bật/tắt từng frame riêng để kiểm tra quan hệ cha-con.

Các lỗi thường gặp (nên nhớ trước khi gặp)

"Lookup would require extrapolation into the future/past": bạn hỏi transform tại một thời điểm mà buffer chưa/không còn dữ liệu. Dùng `rclpy.time.Time()` (nghĩa là "thời điểm mới nhất có sẵn") thay vì timestamp cụ thể khi mới học.

"Could not find a connection between frames": hai frame không nằm chung một cây (do thiếu node publish transform trung gian, hoặc node đó chưa chạy).

Multiple parents cho cùng 1 frame: hai node khác nhau publish transform có cùng `child_frame_id` nhưng khác `frame_id` cha → chỉ được publish 1 nguồn transform cho mỗi frame.

Quên `header.stamp` hoặc set sai giờ hệ thống → transform bị coi là "quá cũ" hoặc "trong tương lai".

Bài tập tổng hợp giai đoạn 2

Viết static broadcaster cho `base_link → laser_frame` (cảm biến lidar đặt cố định).

Viết dynamic broadcaster giả lập robot đi theo hình vuông thay vì hình tròn.

Viết node listener tính khoảng cách Euclid giữa `map` và `base_link` mỗi giây (cần thêm 1 static/dynamic broadcaster cho `map → odom`).

Chạy `view_frames`, mở PDF, xác nhận cây transform đúng như bạn kỳ vọng (`map → odom → base_link → laser_frame`).

Mở RViz2, quan sát các frame di chuyển theo thời gian thực.

Sau khi xong 2 giai đoạn này

Bạn đã có đủ nền tảng để bước tiếp sang **URDF** (mô tả hình dạng vật lý robot, sinh ra transform tree tự động qua `robot_state_publisher`) và **Nav2** (dùng toàn bộ Topic/Service/Action/TF vừa học để điều hướng robot tự động). Hai giai đoạn này không phải lý thuyết suông — hãy chạy được tất cả các đoạn code trên bằng tay trước khi đi tiếp.